

Aula 3 - Padrões de Codificação e Boas Práticas em Java

Duração: 2 horas

Objetivos:

- Aplicar convenções de codificação (Java Code Conventions)
- Utilizar ferramentas de análise estática (Checkstyle, PMD)
- Praticar refatoração com foco em legibilidade e manutenibilidade

Parte Teórica (45 min)

1. Importância dos Padrões de Codificação

- **Consistência:** Código uniforme em equipes
- **Legibilidade:** Facilita manutenção e onboarding
- **Prevenção de bugs:** Detecção precoce de más práticas

2. Ferramentas de Análise Estática

Ferramenta	Função	Exemplo de Regra
Checkstyle	Verifica estilo de código	Máx. 80 caracteres por linha
PMD	Detecta más práticas	Evitar if-else aninhados
SonarQube	Análise completa de qualidade	Cobertura mínima de testes: 80%

3. Boas Práticas em Java

- **Nomenclatura:**
 - Classes: `PascalCase` (`CalculadoraSalario`)
 - Métodos: `camelCase` (`calcularImposto()`)
 - Constantes: `UPPER_CASE` (`TAXA_JUROS`)
- **Organização:**
 - 1 responsabilidade por classe/método

- o Evitar métodos com > 3 parâmetros

Parte Prática (1h15min)

Atividade 1: Configurando Checkstyle no Projeto

Passo a passo:

1. Adicione o plugin ao `pom.xml` :

xml

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-checkstyle-plugin</artifactId>
  <version>3.2.1</version>
  <configuration>
    <configLocation>google_checks.xml</configLocation>
  </configuration>
</plugin>
```

2. Execute a análise:

bash

```
mvn checkstyle:check
```

3. Corrija os erros reportados (exemplo comum):

java

```
// Antes
public void calc(int x,int y){...}

// Depois
public void calcularArea(int largura, int altura) {...}
```

Atividade 2: Refatoração Guiada por PMD

Código inicial (problemas):

java

```

public class Relatorio {
    public void gerar(List<Cliente> c) {
        if(c != null) {
            for(Cliente cli : c) {
                if(cli.getIdade() > 18) {
                    System.out.println(cli.getNome());
                }
            }
        }
    }
}

```

Exercício para alunos:

1. Instale o PMD no projeto (via Maven/Gradle)
2. Refatore considerando:
 - o Nomes descritivos
 - o Extração de métodos
 - o Remoção de nested if

Solução sugerida:

java

```

public class RelatorioClientes {
    public void imprimirClientesMaioresDeIdade(List<Cliente> clientes) {
        if (clientes == null) return;

        clientes.stream()
            .filter(cliente -> cliente.getIdade() > 18)
            .forEach(cliente -> System.out.println(cliente.getNome()));
    }
}

```

Desafio Prático (25 min)

Contexto: Refatore o código abaixo aplicando:

1. Checkstyle (nomenclatura)
2. PMD (complexidade)
3. Princípios SOLID

java

```
class Pagamento {  
    void processar(String t, double v) {  
        if(t.equals("CREDITO")) {  
            // 20 linhas de código  
        } else if(t.equals("BOLETO")) {  
            // 15 linhas de código  
        }  
    }  
}
```

Dicas:

- Use Strategy Pattern para tipos de pagamento
- Extraia métodos curtos
- Adicione validações

Encerramento

- **Discussão:** Qual ferramenta foi mais útil? Por quê?
- **Próxima aula:** Princípios SOLID em Java (Aula 4)

Tarefa:

- Configurar SonarQube local (docker-compose)

Recursos:

- [Google Java Style Guide](#)
- [PMD Rules](#)

Prontos para deixar seu código profissional? Dúvidas? É só perguntar!  